# Client Bug & Performance Report

Oct 19, 2026 · @Adwaith Anand Sr

The client has 30 bugs and 13 performance problems in player and playlist code. Most trace to one gap: the player queue is identified only by playlist ID, so shuffle, downloads and removals can act on the wrong queue. Fixing that first removes or simplifies seven of the bugs below (A1, A2, A3, A6, B1, B2, C1) and makes the other patches smaller.

Severity: **High** = wrong playback or data in a common path, or constant heavy work. **Medium** = reproducible edge case. **Low** = polish. References are `file:line` under `client/src`; findings come from reading the code, not from running the app.

## Fix order

Start with the queue identity change in `store/player.js`, because steps 1 to 5 all edit the same three functions (`changePlaylistAndPlay`, `appendToQueue`, `next`). Doing them in this order avoids rewriting each patch twice. Steps 6 to 9 do not touch the store and can run in parallel once step 1 lands.

| Step | Patch | Resolves | Re-test after |
|---|---|---|---|
| 1 | Give the queue an identity. Store `queueKey` (`playlistId`, `isRandom`, `seed`) and `isLocalPlaylist` when `changePlaylistAndPlay` runs. Pass the same key from `getPlaylistSongs` into `appendToQueue`, key `playlistControllers` by it, and compare it in `shouldUpdateQueue`. | A1, A2, A3, half of A6 | Turn Shuffle on, Reshuffle, tap a song: playback order matches the list. |

| Step | Patch | Resolves | Re-test after |
|---|---|---|---|
| 2 | Use the local flag and honour shuffle. Replace both `queue[0].isLocal` checks with `isLocalPlaylist`. Stop `changePlaylistAndPlay` overwriting `tracksOverride`. Use Fisher-Yates. | B1, B2 | Play a cloud playlist whose first song is downloaded, press Next at the end. Shuffle Play in Downloads. |
| 3 | Make queue writes safe and sync loop mode in one place. Add a `mutateQueue` helper that reads `get()` inside `set` and bails if the key changed. Add one `syncLoopMode()` called after append, repeat change, controller update and download events. Wrap player teardown in try/catch. | A4, A7, B3, B5 | Repeat-all on a playlist with more than one page. Downloads finishing during playback. |
| 4 | Scope removals and deletes. `removeFromQueue(ids, key)` ignores other playlists. Call it after permanent delete. Prefix-match cache updates. Keep the cloud URL in download metadata. | C1, C2, C3, B4 | Remove a song from playlist A while playing B. Delete a downloaded song that is playing. |
| 5 | Pagination at the end of the queue. In queue mode, Next waits for `fetchNextPage` before wrapping. Controllers become getters and are cleared on unmount. | A5, A6 | Throttle the network, play the last loaded track. |
| 6 | Downloads screens (separate patch, independent). Stop the 2 s polling, throttle progress updates, select ids only. | P1, P2, B6 | Start a 20-song download with Downloads open. |
| 7 | Full-screen player. Reset lyrics state in `trackChanged`, split `VideoBackground`, fix the shadow and colour races. | D1, D2, D4, P3, P7 | Let a song with lyrics end while lyrics are open. |

| Step | Patch | Resolves | Re-test after |
|---|---|---|---|
| 8 | List rendering. Map items once with `select`, mount `LongPressOptions` on demand, add image recycling keys, drop duplicate requests. | P5, P6, P8, P9, P11, P12 | Scroll a 200-song playlist, toggle a favourite. |
| 9 | Server-side. Drop `lyrics` from list responses, implement shuffle for Favourites and Artist playlists. | P4, C4 | Needs a backend change and a lyrics-on-demand route. |

Quick wins you can do at any time, each a few lines: D3 (regex escape), D2 (shadow fallback), D5, D6, C6, P10, P13.

## A. Queue, shuffle and pagination

The queue is matched to a screen by playlist ID alone, so anything that fetches the same playlist in a different order corrupts it.

- **A1 · High · `controllers/playlists/fetch.js:46`, `store/player.js:533`.** `getPlaylistSongs` calls `appendToQueue` inside the query function, and `appendToQueue` only checks `playlistId === currentPlaylistId`. Toggling Shuffle, Reshuffle, or switching back to the normal view fetches page 1 of a different order and appends its unseen songs to the end of the playing queue and the native playlist. Fix: pass `random` and `seed` into `appendToQueue` and ignore a mismatch with the stored queue key (step 1).
- **A2 · High · `store/player.js:472`.** `shouldUpdateQueue` compares playlist ID, shuffle flag and length but not `randomSeed`. After Reshuffle, tapping a song reuses the old queue when lengths match, so playback follows the old shuffle while the list shows the new one. A1 currently hides this by changing the length. Fix both together.
- **A3 · Medium · `store/player.js:477`.** Length equality stands in for "same list". Any drift forces a full player teardown and restarts the tapped song: a favourite removed from Favourites (list refetches to n-1), pull-to-refresh (list resets to one page), duplicate IDs across pages (Home dedupes for display, but the queue is built from the raw pages). Tapping the song that is already playing restarts it. Fix: compare the queue key and ID prefix, and only resume if the track is current.
- **A4 · Medium · `store/player.js:88-98, 551`.** `getEffectiveLoopMode` returns `none` while the controller says `hasNextPage`. `appendToQueue` sets `player.loop` before the screen's effect has updated that controller, and `setPlaylistController` never

recomputes it. After the last page loads, loop stays `none`, so repeat-all stops at the end of the last song instead of wrapping. Fix: call a `syncLoopMode()` from `setPlaylistController`.

- **A5 · Medium · `store/player.js:292, 333-336, 160`.** `shouldFetchPage` is false in the default `queue` repeat mode, so Next on the last loaded song wraps to index 0 even when more pages exist. Only the three-songs-left prefetch protects this. `hasEnded` is set on every finish but nothing reads it, so a slow or failed prefetch ends playback silently. Fix: in queue mode, wait for `fetchNextPage` before wrapping.
- **A6 · Medium · `PlaylistSongs.jsx:187-193`, `Home.jsx:41-47`, `store/player.js:78-86, 197`.** The controller stored per playlist is a snapshot of `fetchNextPage`, `hasNextPage` and `isFetchingNextPage` from the screen's last render. It is never removed on unmount, shares one slot between the shuffled and normal queries, and its booleans go stale. The prefetch can start a fetch while one is running, and React Query's `fetchNextPage` cancels and restarts the in-flight request by default; the axios call is not aborted, so its append still runs. Fix: store getters, key by queue key, clear on unmount.
- **A7 · Medium · `store/player.js:400, 487-526`.** `changePlaylistAndPlay` has no error handling. After `player.destroy()` the store still points at the destroyed player until the new one is set, so a throw in `createAudioPlaylist` leaves playPause calling a dead object. It also reads `player`, `queue` and `currentPlaylistId` before the awaits and uses them after. Fix: `set({ player: null })` right after destroy, wrap in try/catch, re-read `get()` after awaits.
- **A8 · Low · `store/player.js` (several).** A `trackId` that is not found silently plays index 0 (467). `prev()` at index 0 with repeat off does nothing instead of restarting the song (382-387). Next on a one-song queue does nothing because `playByIndex` returns early when the index is current and playing (226). `isWaitingForDownload` is never initialised and `playbackRate` is not reset in `resetValues` (656-674).
- **A9 · Low · `store/player.js:186-187, 241-242`, `services/widgetService.js:117-142`.** `playByIndex` updates the lock screen and widget, then the `trackChanged` event should do both again (assumed, see Limits). Rapid skips also start overlapping artwork downloads that write the same `current_artwork.jpg`. Fix: update only from `trackChanged`.

## B. Downloads and local playback

The store guesses whether a queue is local from its first song, and several handlers write back a queue snapshot taken before an await.

- **B1 · High · `store/player.js:295, 686`.** `queue[0].isLocal` decides whether the queue is a downloaded playlist. `normalizeTrack` (12-50) marks any downloaded cloud song as `isLocal`, so a cloud playlist whose first song is downloaded is treated as local. Next at the end of the queue then returns early waiting for a download (323-330) when one is running for that playlist, and `onLocalSongDownloaded` can replace the cloud queue with the downloaded list and add mismatched native sources. Fix: store `isLocalPlaylist` from the `isLocal` argument (step 2).
- **B2 · High · `DownloadedPlaylistSongs.jsx:90-99`, `store/player.js:412-424`.** Shuffle Play in Downloads sends a shuffled `tracksOverride`, but `changePlaylistAndPlay` replaces `tracks` with `getDownloadedSongs` order whenever `isLocal` is set. Only the first song is random; the rest play in stored order. The shuffle itself (`sort(() => Math.random() - 0.5)`, also at `PlaylistSongs.jsx:165`) is biased. Fix: honour the override and use Fisher-Yates.
- **B3 · Medium · `store/player.js:88-98, 323-330`.** This is the first item in `client/todo.txt`. When the downloaded songs end while more are still downloading, the native loop is `all` and wraps to the first song. `next()` handles a manual skip but the natural end is never intercepted. Fix: set loop to `none` while the current local playlist has pending downloads and continue when the next one is saved.
- **B4 · Medium · `store/player.js:760-808`, `PopUpOptions.jsx:104-112`.** Download metadata stores `url: localUrl` and no cloud URL (`downloadService.js:576-582`). When a song from a local playlist is deleted, `onLocalSongDeleted` reverts it to `track.cloudUrl || track.url`, which is the deleted file path. Removing a download also never calls `removeFromQueue`, so the song stays queued with a dead source. Fix: save the original URL in metadata, and remove the track from the queue if there is none.
- **B5 · Medium · `store/player.js:537-556, 676-758, 760-808`.** `appendToQueue`, `onLocalSongDownloaded` and `onLocalSongDeleted` copy `queue` and `currentTrackIndex`, await, then call `set({ queue })` from the copy. This overwrites any change made meanwhile (favourite toggle, removal, another append) and decides which native source to swap from a stale index. The window is small once the metadata

cache is loaded, because the awaits are microtask-only, but it is open on cold start and breaks if any await becomes real I/O. Fix: compute inside `set(state => ...)` or serialise queue mutations.

- **B6 · Medium ·** **`DownloadedPlaylistSongs.jsx:29-33, 48-50`**, **`services/downloads/downloadService.js:64-67, 234, 599`**. The screen only updates its list when the length differs. `getDownloadedSongs` returns the live cached array, which `downloadService` mutates with `push` and `splice`, so the previous and fetched values can be the same array and never differ. Deleting one song and adding another never shows. Fix: return copies and compare an ID signature.
- **B7 · Low ·** **`controllers/playlists/handleToggleFavourite.js:41-45`**, **`services/widgetService.js:274-291`**. The toggle updates the query cache and `queue` but not `currentTrack`. Toggling from the widget leaves `track.isFav` stale, and the full-screen double-tap ( `TrackControllerFullView.jsx:228` ) then decides from the stale value and can un-like a liked song.

## C. Playlist management

Remove and delete actions update the player queue without checking which playlist is playing, and cached shuffled lists are skipped.

- **C1 · High ·** **`controllers/playlists/removeSong.js:28, 69`**, **`PopUpOptions.jsx:128, 270`**. `removeFromQueue(songId)` runs after every successful removal without comparing the edited playlist to the playing one. Removing song S from playlist A while playing playlist B, which also contains S, pulls S out of B's queue. If S is the current track, playback jumps to the next song. Fix: pass the playlist key and ignore a mismatch.
- **C2 · Medium ·** **`PopUpOptions.jsx:200-206, 242-245`**, **`ListHeader.jsx:90-96`**, **`Search.jsx:102`**. Permanent delete from Home or Search never calls `removeFromQueue`, so deleted songs stay queued. All four paths then call `queryClient.invalidateQueries()` with no key, which refetches every page of every active list. Fix: remove from the queue, and invalidate only the affected keys.
- **C3 · Medium ·** **`removeSong.js:30, 72`**. The cache update targets `[playlistId]` exactly. Shuffled results live under `[playlistId, "random", seed]`, so a removed song stays visible in the shuffled view until it refetches. Fix: `setQueriesData({ queryKey: [playlistId] }, ...)` matches by prefix.

- **C4 · Medium · `server/handlers/playlists/songs.playlist.js`, `ListHeader.jsx:46-50`.** The server checks the artist, special and favourites branches before `isRandom`, so `random` is ignored for them. The client still shows Shuffle, Reshuffle and "Shuffled" for any ID except Home and Search, and Shuffle Play plays in normal order. Fix: shuffle on the client for those lists, or hide the buttons.
- **C5 · Low · `ListHeader.jsx:54-57`.** Tapping the title scrolls to `currentTrackIndex`, which is an index in the player queue, not in this list. It lands on the wrong row, or nowhere, when the playing queue is another playlist or a shuffled order. Fix: check `currentPlaylistId` and look up `currentTrackId` in the list data.
- **C6 · Low · `DownloadedPlaylistSongs.jsx:103-111`.** The header shows a Shuffle toggle for downloaded playlists but no `onToggleRandom` handler is passed, so the button does nothing.
- **C7 · Low · `Search.jsx:155-165`.** Search results play through `ID = SEARCH-<text>`, but search never calls `appendToQueue` or registers a controller. The queue stops at the pages loaded when the song was tapped, and later taps hit the length mismatch in A3.

## D. Full-screen player and list UI

Per-track UI state is reset by button handlers, not by the track change itself.

- **D1 · Medium · `TrackControllerFullView.jsx:43-45, 121`, `LyricsView.jsx:28, 57`, `store/player.js:173-203`.** `resetShowLyrics` runs only from a list tap and the Next and Previous buttons, not from the `trackChanged` listener or the mini-player swipes. If lyrics are open and a song without lyrics starts, `showLyrics` stays true, `LyricsView` renders nothing, and the pan gesture is disabled (`.enabled(!showLyrics)`), so swipe-down to close stops working. The lyric buttons are hidden because the new track has no lyrics, so there is nothing to toggle off. `lastIndexRef` and `currentLyricIndex` also carry over between tracks. Fix: reset in `trackChanged`.
- **D2 · Medium · `components/fullView/CoverArtwork.jsx:57`.** The shadow is built as `` `0px 30px 100px ${shadowColor}b0` ``. On the first render `colors` is null, so `shadowColor` is undefined and the value becomes `undefinedb0`. A 100 px blur on an 85%-width view is also expensive on Android. Fix: build the style only when a colour exists, with a smaller radius.
- **D3 · Medium · `components/HighlightedTitle.jsx:20`.** The search text goes straight into `new RegExp(...)`. Any search containing an unbalanced `(` or `[` throws a SyntaxError and crashes the Search screen. Fix: escape the input.

- **D4 · Low · `TrackControllerMinView.jsx:30-42`, `TrackControllerFullView.jsx:54-69`.** Both call `getColors(...).then(setColors)` with no cancellation, so a slow result from a skipped track can overwrite the current colours. A track with no cover keeps the previous track's colours. The two components also extract separately with different cache keys (`url` and `trackId`). Fix: extract once, store the result on the track.
- **D5 · Low · `components/fullView/OptionsContainer.jsx:20`.** `artistIndex` is component state and survives track changes. After cycling to the third artist, a track with one artist shows an empty chip. Fix: give the component `key={trackId}`.
- **D6 · Low · `components/ListItem.jsx:125-136`.** The equalizer animation renders for the current track whether or not it is playing. Fix: select `currentTrackId === id && isPlaying`.
- **D7 · Low · `TrackControllerFullView.jsx:222-233`.** The double-tap worklet captures the whole `track` object, which carries the lyrics arrays, so Reanimated copies it to the UI runtime whenever the gesture is rebuilt. Read `isFav` in the JS callback instead. I did not profile this; check it before spending time.

## Performance and re-renders

The two biggest costs are on the Downloads screens, which do constant background work. None of the fixes below need `memo`, `useMemo` or `useCallback`; the compiler handles those, and the causes here are changing props, subscriptions and side effects.

- **P1 · High · `DownloadedPlaylists.jsx:17-44`.** A 2-second `setInterval` re-reads every playlist's songs and always calls `setPlaylists` with a new array of new objects. While the screen is focused the whole list re-renders every 2 seconds and each card rebuilds its image source. Runs can also overlap. Fix: load on focus and refresh from a download-completed event, or compare a signature before setting state.
- **P2 · High · `store/appState.store.js:137-163`, `services/downloads/downloadService.js:339-361`, `DownloadedPlaylistSongs.jsx:24, 37-56`.** Each progress callback is unthrottled and rebuilds `downloadingPlaylists` (a map over the playlist) and `downloadingSongs`. The screen selects the whole array, so every tick re-renders it. The effect then sees a new array, reads metadata, and sets state again, which is two renders and a metadata lookup per tick. Fix: update only on a 1% or 250 ms change, select IDs only, and keep progress in `downloadingSongs`, where `ListItem` already reads it.

- **P3 · Medium · `components/fullView/VideoBackground.jsx:7-9, 23-26, 50`.** The component subscribes to `position`, so it re-renders twice a second while music plays with the full-screen player open, even when video is off. `useVideoPlayer(videoUrl)` also runs for every track with a video before the `return null` at line 50. That likely starts loading video data you never show; confirm in a network profile. Fix: move the video into a child that mounts only when `showVideo && videoUrl`, and read position inside it.
- **P4 · Medium · `server/handlers/playlists/songs.playlist.js:6`, `downloadService.js:48-57`.** `MUSIC_SELECT_FIELDS` includes `lyrics` and `lyricsAsText`, so every song in every list page carries its full lyrics. They are held in the query cache, the queue, multi-select and the download metadata, and `saveMeta` serialises the entire `meta.json` after each downloaded song. Fix: return lyrics only for the current track from a separate route.
- **P5 · Medium · `components/playlists/ListItem.jsx:30, 146-152`.** Every playlist card renders a `LongPressOptions` modal, each with two shared values and two animated styles. Each card also subscribes to the whole `selectedSongs` array. The Playlists tab stays mounted under `PlaylistSongs`, so every song selection re-renders all its cards. Fix: mount the sheet only when `showOptions` is true, and read `selectedSongs` with `getState()` in the Add handler.
- **P6 · Medium · `PlaylistSongs.jsx:201-211`, `Home.jsx:49-60`, `Search.jsx:57-65`.** Every song is re-mapped to `{ id, _id, artwork, cover, ...rest }` whenever `data` changes. Each page fetch, favourite toggle or refetch gives all mounted rows a new `item` object, so they all re-render. The compiler cannot avoid this because the props really changed. Fix: map once with the query's `select` so structural sharing keeps unchanged items, or return those fields from the server.
- **P7 · Medium · `components/fullView/LyricRenderItem.jsx:14-15`, `LyricsView.jsx:25`.** Every lyric line subscribes to `currentLyricIndex`, so each line change re-renders every mounted line. `LyricsView` also re-renders at 2 Hz on `position`. Fix: select a boolean ( `currentLyricIndex === index - 1 && showSyncedLyric` ) so only two lines update.
- **P8 · Low · `PlaylistSongs.jsx:122-149, 179-185`, `Playlists.jsx:53-56`.** Shuffle Play requests page 1 itself while the new query key from `setSeed` and `setIsRandom` requests the same page. Pull-to-refresh calls `resetQueries` then `refetch`, a second request. Fix: use `queryClient.fetchInfiniteQuery` for Shuffle Play and drop the extra `refetch`.

- **P9 · Low · `components/ListItem.jsx:93-103`.** List images use `transition={1000}` with no `recyclingKey`, so a recycled FlashList row cross-fades from the placeholder for a full second. Playlist cards use 500 ms with a parallax image. Fix: add `recyclingKey={item.id}` and a transition of 200 ms or less.
- **P10 · Low · `components/fullView/SliderContainer.jsx:69-77`.** While dragging, `runOnJS(setSeekTime)` fires every frame, about 60 React renders a second, but only whole seconds are shown. Fix: call it only when `Math.floor(time)` changes.
- **P11 · Low · `store/player.js:139-170`.** `position`, `progress` and `duration` live in the same store as the queue, so every 500 ms tick notifies all subscribers. Each selector is cheap but every `ListItem` has several. Fix: move them to their own store, or raise `updateInterval` unless the slider or lyrics are visible.
- **P12 · Low · `controllers/playlists/handleToggleFavourite.js:21-38`.** `setQueriesData({ predicate: () => true })` walks every cached query, including each search and shuffle seed, on every favourite tap. Fix: target the open playlist and Favourites.
- **P13 · Low · Eight `FlashList` usages, `Home.jsx:88`.** `estimatedItemSize` and `initialNumToRender` are no longer needed by FlashList 2.x as far as I know; check the v2 migration notes before removing them. `DownloadedPlaylistSongs.jsx:113` has no `keyExtractor`.

Checked and fine, so do not chase these: the slider's progress path (shared value fed by `usePlayer.subscribe`), the `CurrentTimeDisplay` leaf, and the selectors in `ListItem` and the controls, which all return primitives.

## Limits of this audit

This is a code reading only: I did not run the app, attach a profiler, or run the test suite, so each item needs a quick on-device check before and after its patch.

- **React Compiler not verified.** The package registry was unreachable from my environment, so I could not run the compiler's healthcheck. Run `npx react-compiler-healthcheck` and your `react-compiler` lint rule to see which components it skips. First suspects: `ListHeader.jsx` and `playlists/ListItem.jsx` (they call `scrollY.interpolate` during render), `ItemSeparator` defined inside `PlaylistSongs` at line 237 (if it is not memoised, FlashList remounts every separator on each render), and inline `require()` calls in `player.js`, `PopUpOptions.jsx` and `ListHeader.jsx`.

- **Native behaviour assumed.** I assumed `skipTo` fires `trackChanged` (A9), that loop `none` stops at the end of the queue (A4, B3), and that `didJustFinish` arrives once. Confirm these three on a device.
- **Read in full:** the stores, playlist and search screens, controllers, `downloadService`, `widgetService`, the full-screen player components, `PopUpOptions`, `ListHeader`.
- **Skimmed:** `SleepTimerPopup` and `WheelPicker` (timer selection only).
- **Not read:** `AdminPanel`, `Youtube`, `ColorExtractor`, `index.jsx`, `Loader`, `DestinationPickerModal`, `DownloadOptionsModal`, `AddPlaylist`, `MusicWidget.tsx`, and the server except `songs.playlist.js`.
- **Dead code:** `components/Navbar.jsx` is not imported anywhere, and the append subscriber in `services/queryClient.js` is commented out.